Entropia i segmentacja plików binarnych

Sprawozdanie PRIR – Projekt 16 (C05)

Mariusz Gosławski, Krzysztof Śliwiński, Informatyka, studia stacjonarne, specjalność Cyberbezpieczeństwo

Informacje o dokumencie

Pole	Wartość
Przedmiot	Programowanie równoległe i rozproszone
Temat	nr 16 – Entropia i segmentacja plików binarnych (C05)
Zespół	MG, KŚ
Rok akademicki	2026/2027
Repozytorium	https://github.com/NoMariusz/project-prir-entropy
Wersja dokumentu	0.2 – szkic

Oznaczenia: **(MG)**, **(KŚ)** – kto odpowiada za rozdział; **Co tu wpisać:** – wskazówka do usunięcia; **Przykładowa treść:** – tekst do poprawienia; **[DO UZUPEŁNIENIA]** – brakujące dane.

1. Wstęp (MG)

1.1 Opis problemu

Przykładowa treść: Projekt dotyczy programu, który liczy entropię rozkładu bajtów w kolejnych oknach pliku. Wynikiem jest wykres entropii względem pozycji w pliku oraz lista miejsc, w których entropia zmienia się bardziej niż zadany próg. Takie zmiany mogą wskazywać granice obszarów o innej budowie, np. tekstu, danych skompresowanych lub losowych, ale program nie rozpoznaje, czym są te obszary.

Sama wysoka entropia nie dowodzi, że plik jest złośliwy lub zaszyfrowany. Oznacza tylko, że bajty w oknie są rozłożone równomiernie, co jest typowe także dla zwykłych plików ZIP czy JPEG.

1.2 Entropia Shannona

Dla okna o długości n bajtów liczymy histogram $c_0 \dots c_{255}$, czyli ile razy wystąpił każdy z 256 możliwych bajtów. Udział bajtu i to $p_i = c_i / n$. Entropia okna:

```
H = -\sum_{i=0}^{255} p_i \log_2 p_i
```

Składniki z $p_i = 0$ pomijamy. Wynik jest w bitach na bajt, od 0 do 8.

Zawartość okna	Entropia (bit/bajt)

Zawartość okna	Entropia (bit/bajt)
jeden powtarzany bajt, np. same zera	0
dwa różne bajty, każdy w połowie okna	1
16 różnych bajtów, każdy równie często	4
zwykły tekst	ok. 4–5
dane skompresowane lub zaszyfrowane	blisko 8

2. Cel i zakres projektu (MG)

2.1 Cel projektu

Przykładowa treść: Celem projektu jest program, który liczy entropię w kolejnych oknach pliku. Program powstaje w czterech wersjach: sekwencyjnej (referencyjnej), OpenMP, MPI oraz CUDA lub OpenCL. Wszystkie wersje przetwarzają te same pliki i dają ten sam wynik.

Pytania, na które odpowiada projekt (odpowiedzi w rozdziale 8):

1. Czy wszystkie wersje dają ten sam wynik co wersja referencyjna?
2. Ile przyspieszają wersje równoległe, gdy doliczymy przesyłanie danych i komunikację?
3. Czy zmiany entropi wykazują granice doklejanych segmentów do pliku?

2.2 Zakres

- Program wiersza poleceń w czterech wersjach: referencyjnej, OpenMP, MPI oraz CUDA lub OpenCL.
- Okna o stałej długości i stałym kroku, stały próg zmiany.
- Rozszerzenie dla zespołu dwuosobowego: przyrostowa aktualizacja histogramu (opis w 3.4).
- Generator plików testowych i testy poprawności.
- Pomiary czasu i wykresy.

Poza zakresem: rozpoznawanie rodzaju danych w oknie, ocena, czy plik jest złośliwy, oraz interfejs graficzny.

2.3 Podział pracy

Zadanie	Osoba
Wersja referencyjna (sekwencyjna)	MG
Wersja OpenMP	MG
Rozszerzenie (przyrostowy histogram)	MG
Generator plików testowych	MG
Wersja MPI	KŚ

Zadanie	Osoba
Wersja CUDA lub OpenCL	KŚ
Pomiary	każdy dla swoich wersji
Wykresy	KŚ
README.md w repozytorium	MG
Sprawozdanie: rozdziały 1–3	MG
Sprawozdanie: rozdział 4	każdy dla swoich wersji
Sprawozdanie: rozdziały 5–9, w tym podsumowanie	KŚ
Prezentacja	KŚ

Obie osoby muszą rozumieć całe rozwiązanie, nie tylko swoją część.

3. Założenia wspólne (MG)

Wszystkie wersje mają te same parametry, reguły i format wyniku, więc wyniki można porównać okno po oknie.

3.1 Dane wejściowe i parametry

Wejściem jest dowolny plik czytany jako ciąg bajtów: pliki z generatora (opis w 5.1) oraz 1–2 prawdziwe pliki. Każda wersja to osobny program z tymi samymi parametrami, np.:

```
entropy_seq --input plik.bin --window 4096 --step 4096 --threshold 1.0
```

Parametr	Znaczenie	Domyślnie
--input	ścieżka do pliku	–
--window	długość okna w bajtach	4096
--step	krok okna w bajtach	4096
--threshold	próg zmiany entropii (bit/bajt)	1.0
--mode	full – histogram od zera, incremental – przyrostowo (3.4)	full
--out	katalog na wyniki	bieżący

Liczbę wątków, procesów lub rozmiar bloku GPU opisuje rozdział 4 przy danej wersji.

3.2 Reguły liczenia okien

- Okno nr k zaczyna się w bajcie k · krok i ma długość okna.
- Entropię liczymy w typie double; składniki z $p_i = 0$ pomijamy.
- Ostatnie niepełne okno: entropia z dostępnych bajtów, okno oznaczone jako niepełne i nieużywane do wykrywania zmian.
- Pusty plik daje pusty wynik.

3.3 Wykrywanie zmian

- Zmiana występuje między oknem k a k + 1, gdy |H(k+1) − H(k)| > próg. Pozycją zmiany jest początek okna k + 1.
- Przy nachodzących oknach (krok mniejszy od okna) entropia zmienia się łagodniej; wpływ tego ograniczamy doborem progu.
- Wszystkie wersje używają tej samej prostej funkcji na gotowej tablicy entropii. Zrównoleglamy tylko histogramy i entropię.

3.4 Przyrostowa aktualizacja histogramu (rozszerzenie)

Gdy okno przesuwa się o krok mniejszy od swojej długości, część bajtów zostaje w oknie. Zamiast liczyć histogram od zera, odejmujemy bajty, które wypadły z okna, i dodajemy te, które weszły. Entropię liczymy dalej z całego histogramu, więc wynik musi być identyczny jak przy liczeniu od zera. Tryb włącza parametr `--mode incremental`.

3.5 Format wyniku

- `entropia.csv` – jeden wiersz na okno:

```
okno,poczatek,dlugosc,entropia,niepelne
0,0,4096,4.512,0
1,4096,4096,7.981,0
```

- `zmiany.csv` – (lista zmian przekraczających ustalony próg) pozycja zmiany w bajtach, entropia przed i po.
- `czasy.csv` – czasy etapów: odczyt, obliczenia, zapis.

4. Implementacje (MG, KŚ)

Każdą wersję opisuje osoba, która ją wykonała. Opis ma ten sam układ: język i narzędzia, podział pracy, synchronizacja i komunikacja, najważniejsze decyzje. Pełny kod jest w repozytorium.

4.1 Wersja referencyjna (MG)

Język: C++17, kompilator gcc 13.

Co tu wpisać:

- Pętla po oknach: zerowanie histogramu, zliczanie bajtów, entropia.
- Tryb przyrostowy (3.4): jak program przechodzi z okna do okna.
- Sposób odczytu pliku: cały do pamięci czy porcjami.

4.2 Wersja OpenMP (MG)

Język: C++17, kompilator gcc 13.

Co tu wpisać:

- Okna rozdzielone między wątki (`#pragma omp parallel for`); wybrany przydział i powód.
- Każdy wątek ma własny histogram, a każde okno zapisuje wynik do własnej komórki tablicy, więc nie ma wspólnych liczników ani operacji atomowych.

- Tryb przyrostowy: każdy wątek dostaje ciągły zakres okien, pierwszy histogram liczy od zera, kolejne przyrostowo.

4.3 Wersja MPI (KŚ)

Język i biblioteka MPI: [DO UZUPEŁNIENIA].

Co tu wpisać:

- Podział pliku: każdy proces dostaje zakres okien i zakładkę (okno − krok bajtów) potrzebną do ostatnich okien.
- Każde okno liczy tylko proces, w którego zakresie leży początek okna; pozycja globalna = przesunięcie fragmentu + pozycja lokalna.
- Rozesłanie danych i zebranie wyników (np. `MPI_Scatterv`, `MPI_Gatherv`); ich koszt mierzony osobno.

4.4 Wersja CUDA lub OpenCL (KŚ)

Technologia i język: [DO UZUPEŁNIENIA].

Co tu wpisać:

- Przypisanie okien do bloków wątków; histogram w pamięci współdzielonej, zliczany operacjami atomowymi.
- Wpływ zawartości pliku: dla samych zer wszystkie wątki zwiększają ten sam licznik (najgorszy przypadek).
- Dane przesyłane na urządzenie raz, z powrotem tylko tablica entropii; transfery mierzone osobno.
- Wybrany rozmiar bloku.

5. Testy poprawności (KŚ)

Przed pomiarami czasu każda wersja musi dać ten sam wynik co wersja referencyjna.

5.1 Pliki testowe

Generator zapisuje użyte ziarno losowania, aby pliki dało się odtworzyć.

Plik	Zawartość	Oczekiwany wynik
stale.bin	same zera	H = 0
wzorzec16.bin	bajty 0–15 powtarzane po kolei	H = 4
losowy.bin	losowe bajty	H blisko 8
segmenty.bin	sklejone: tekst, zera, losowe bajty	zmiany w znanych miejscach
pliki brzegowe	pusty, krótszy niż okno, rozmiar niepodzielny przez krok	poprawne niepełne okno

5.2 Sposób porównania

- Entropia porównywana okno po oknie, z tolerancją [np. 1e-9].
- Listy zmian muszą być takie same.
- Tryb przyrostowy musi dać ten sam wynik co liczenie od zera.
- Okna na granicach fragmentów MPI liczone dokładnie raz.

5.3 Wyniki testów i różnice między wersjami

Wymagania mówią, że różnice algorytmów i dokładności trzeba jawnie opisać.

Wersja	Wynik testów	Różnice względem referencji
Referencyjna – tryb przyrostowy	[OK / największy błąd]	[np. brak]
OpenMP		
OpenMP – tryb przyrostowy		
MPI		
CUDA lub OpenCL		[np. funkcja $\log_2$, kolejność sumowania]

6. Metodyka pomiarów (KŚ)

6.1 Sprzęt i oprogramowanie

Element	Wartość
Procesor (model, rdzenie, wątki)	[DO UZUPEŁNIENIA]
Pamięć RAM	
Karta graficzna	
System operacyjny	
Kompilatory i wersje	gcc 13 (wersje MG), [pozostałe]
Biblioteka MPI, CUDA lub OpenCL – wersje	

Przy MPI na kilku komputerach podać też liczbę węzłów.

6.2 Sposób pomiaru

- Osobno czas obliczeń (histogramy i entropia) i czas całości (z odczytem pliku, transferami, komunikacją i zapisem).
- Jedno uruchomienie na rozgrzewkę, potem co najmniej 5 pomiarów; podajemy medianę i rozrzut (min–max).
- Na GPU synchronizacja przed końcem pomiaru.
- Surowe wyniki zapisujemy w plikach CSV w repozytorium.

6.3 Plan eksperymentów

Co zmieniamy	Wartości (propozycja)
liczba wątków OpenMP lub procesów MPI	1, 2, 4, 8, …
rozmiar pliku	np. 100 MB, 500 MB, 1 GB
rozmiar bloku GPU	np. 128, 256, 512
zawartość pliku (GPU)	same zera, 16 wartości, losowe
krok przy oknie 4096 B (tryb przyrostowy)	4096, 1024, 256

6.4 Miary

Przyspieszenie i efektywność dla OpenMP i MPI, przy tym samym pliku i tej samej metodzie:

```
S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}
```

- Przepustowość w GB/s.
- Udział transferów i komunikacji w czasie całości.
- Dokładność wskazania znanych granic w segmenty.bin (odległość w bajtach).

Uwagi z wymagań: liczby wątków GPU nie używamy we wzorze na efektywność; przyspieszenie GPU podajemy z nazwą wersji odniesienia; wynik MPI z jednego komputera oznaczamy jako lokalny.

7. Wyniki (KŚ)

Każdy podrozdział to wykres lub tabela i 2–3 zdania o tym, co z nich wynika. Sposób tworzenia wykresów wybiera KŚ. Wymagania proszą o dołączenie surowych wyników i skryptów lub plików, z których powstały wykresy.

7.1 Przykładowy wynik programu

Co tu wpisać: wykres entropii dla segmenty.bin z zaznaczonymi wykrytymi zmianami i prawdziwymi granicami. Do tego krótko: błąd entropii (największa różnica względem wartości oczekiwanej, np. 0 dla stale.bin i 4 dla wzorzec16.bin) oraz dokładność lokalizacji znanych granic (odległość w bajtach, liczba pominiętych i fałszywych zmian).

7.2 OpenMP i MPI

Co tu wpisać: czas, przyspieszenie i efektywność w funkcji liczby wątków lub procesów, dla kilku rozmiarów pliku; udział komunikacji MPI.

7.3 CUDA lub OpenCL

Co tu wpisać: czas i GB/s dla różnych rozmiarów pliku i bloku, podział czasu na transfery i obliczenia oraz wpływ zawartości pliku.

7.4 Tryb przyrostowy

Co tu wpisać: czas trybu przyrostowego i liczenia od zera dla kilku kroków przy oknie 4096 B.

7.5 Zestawienie wersji

Najlepszy wynik każdej wersji przy tym samym pliku i tym samym zakresie mierzonych operacji.

Wersja	Konfiguracja	Czas obliczeń	Czas całości	GB/s	Przyspieszenie względem [wersja]
Referencyjna	1 wątek				1,0
OpenMP	[wątki]				
MPI	[procesy, węzły]				
CUDA lub OpenCL	[rozmiar bloku]				

8. Wnioski (KŚ)

Co tu wpisać:

- Odpowiedź na każde z 3 pytań z punktu 2.1, z odwołaniem do wyników z rozdziału 7.
- Kiedy opłaca się tryb przyrostowy. Oczekiwanie do sprawdzenia: zysk pojawia się tylko przy kroku wyraźnie mniejszym od okna; przy domyślnym kroku równym oknu zysku nie ma.
- Wyjaśnienie, gdy wersja równoległa nie przyspiesza (np. odczyt pliku, transfery). Wnioski muszą wynikać z pomiarów także w takim przypadku.
- Ograniczenia projektu, np. jeden komputer, ograniczony rozmiar plików.

9. Podsumowanie (KŚ)

Co tu wpisać: jeden krótki akapit – co zrobiono, najważniejszy wynik liczbowy (np. największe przyspieszenie i dla jakiej wersji) i czy cel z punktu 2.1 został osiągnięty. Przypomnieć, że sama entropia nie rozstrzyga o złośliwości pliku.